Experiment 6

Sequential Logic: Clock Division, LED Patterns, Games and Zynq PS Clocking

Student Seyed Kiarash Eshraghi Naeini (403101383)
Semester Fall 2025 session: 21–22 December 2025
Platform Zynq-7010 (XC7Z010-CLG400-1), 50 MHz clock, Vivado 2019.1

Summary

This experiment moves from combinational to clocked designs. It starts by observing the 50 MHz clock directly, builds a counter-based divider to blink an LED, and uses it for an LED "flow" pattern whose speed is adjusted with push buttons. Two block-design projects follow: a reaction game whose difficulty sets the frequency of its own clock, and the same LED pattern clocked by the ARM processing system (PS) of the Zynq instead of a board oscillator. Two versions of a ping-pong game complete the experiment; the second replaces the divided clock by clock-enable pulses in a single clock domain. All designs were re-verified with self-checking testbenches (7/7 pass); the Vivado reports of the lab session show all timing met. Code: `https://github.com/seyedkiarash/seyedkiarash.github.io/tree/main/labs/logic-circuits-lab/lab06-sequential-zynq`.

1 Part 1 – Seeing the clock

1.1 1a – Clock on an LED

Connecting the clock to an LED (`assign LED[0] = clk`) gives an LED that looks steadily lit at roughly half brightness: it switches 50 million times per second, far beyond what the eye can follow. Vivado warns that a clock net drives an output, which is expected here.

1.2 1b – Blinking LED

A 32-bit counter toggles the LED every 80,000,000 cycles, i.e. every 1.6 s (period 3.2 s). The testbench (half-period shortened to 10 cycles) checks 25 toggles, each exactly one half-period apart.

2 Part 2 – LED flow with adjustable speed

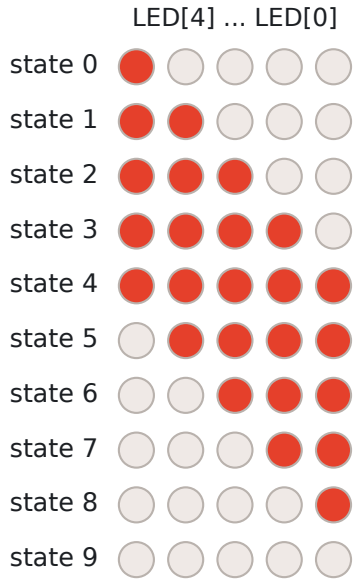
A 10-state counter drives the five board LEDs with a fill-then-empty pattern (Fig. 1). A timer advances the state every max_count + 1 cycles – initially 25,000,001 cycles, about 0.5 s. The value of `max_count` is a register: a rising edge on `button[1]` shifts it left (step time ×2) and on `button[2]` shifts it right (step time /2). Edges are detected by comparing each button with its value one clock earlier, so holding a button changes the speed only once.

Simulation (initial `max_count` = 7): the pattern is correct for 20 consecutive steps; a step lasts 8 cycles, 15 after `button[1]` and 4 after two presses of `button[2]`.

3 Part 3 – Reaction game (block design)

3.1 Design

A light runs across the five LEDs. The player must press `button[2]` while the *middle* LED is lit. A hit raises the level and restarts the run faster; a hit on level 3 wins (all LEDs on); a press on any other LED drops back to level 0. The game was assembled in Vivado IP Integrator from two RTL modules (Fig. 2): the game FSM drives its `mode` (level) output back into the divider that generates its own clock, giving 4, 6, 8 and 10 Hz for levels 0–3.




```
always @(posedge clk) begin           // speed control
    last_b1 <= button[1];
    last_b2 <= button[2];
    if (button[1] & ~last_b1)
        max_count <= max_count << 1;  // slower
    else if (button[2] & ~last_b2)
        max_count <= max_count >> 1;  // faster
end
always @(posedge clk)                 // step timer
    if (t >= max_count) begin
        t     <= 0;
        state <= (state == 9) ? 0 : state + 1;
    end else
        t <= t + 1;
```

Figure 1. Part 2: the ten LED states and the speed-control / timer logic.

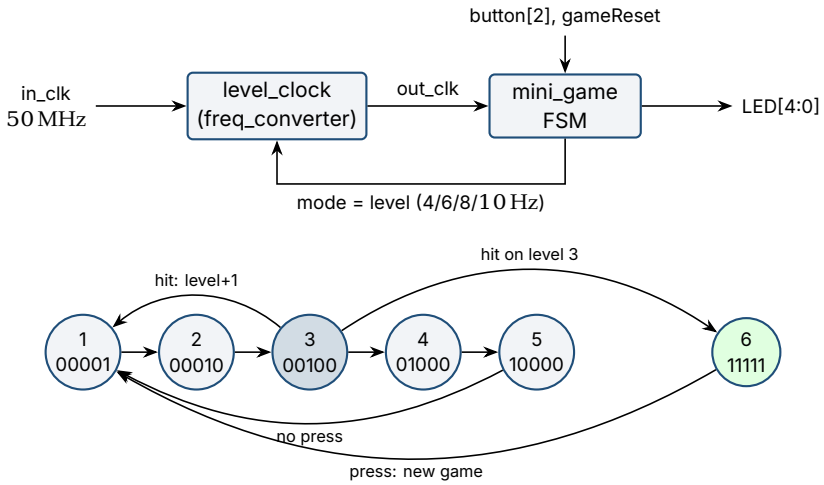


Figure 2. Part 3: block design (top) and game FSM (bottom). A press in states 1, 2, 4 or 5 is a miss (level 0, back to state 1).

3.2 Verification

The FSM was unit-tested with a testbench-driven game clock: four hits on the middle LED reach the win state, and after a new game, a hit followed by a miss returns the level to 0 (Fig. 3).

4 Part 4 – Clocking the logic from the Zynq PS

The Zynq-7000 combines an ARM processing system (PS) with the FPGA fabric (PL). In this part the `processing_system7` IP is configured to output `FCLK_CLK0` = 50 MHz, and the LED-flow module of Part 2 is added to the block design as a module reference clocked by it (Fig. 4). The constraint file therefore contains only the button and LED pins – no clock pin – and Vivado derives the clock constraint (`clk_fpga_0`, 20 ns) from the PS configuration. The PS must be configured (through the boot image or the debugger) before the PL clock runs.

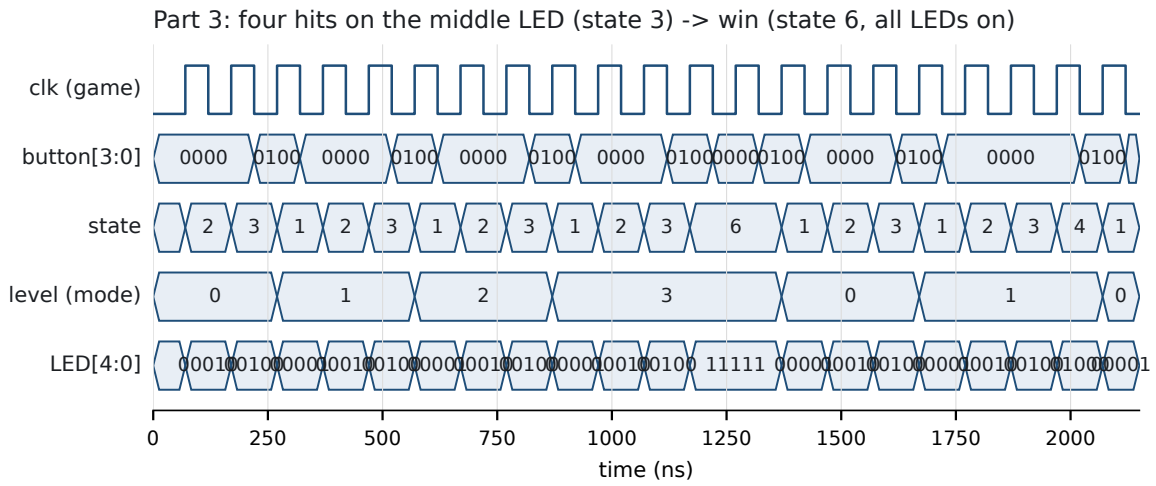


Figure 3. Part 3 simulation: `button[2]` pressed whenever the middle LED (`00100`) is lit.

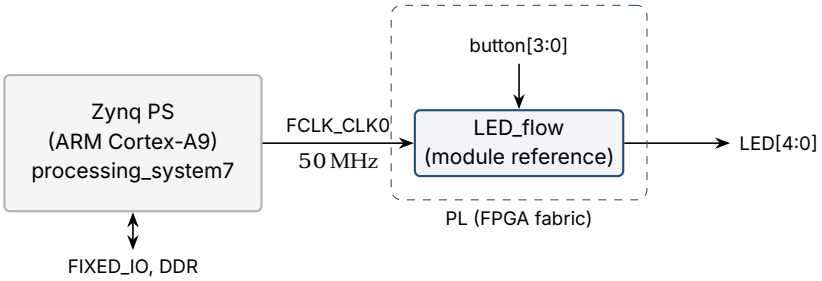


Figure 4. Part 4: the PL design is clocked by the processing system.

5 Bonus tasks – Ping-pong

5.1 A – Ping-pong with a level-controlled clock

A light bounces between LED0 and LED4. When it reaches an end, the player on that side must have pressed his button (`button[3]` at LED4, `button[0]` at LED0) during that step; a return raises the level (= score, shown on the seven-segment display), a miss resets it to 0. As in Part 3 the level selects the game clock, here by halving the divider for every level: $f = 0.37\,\text{Hz} \cdot 2^{\text{level}}$, from 2.7 s per step at level 0 to 5 ms at level 9. Simulation: three returns give a score of 3, and a miss resets it (Fig. 5).

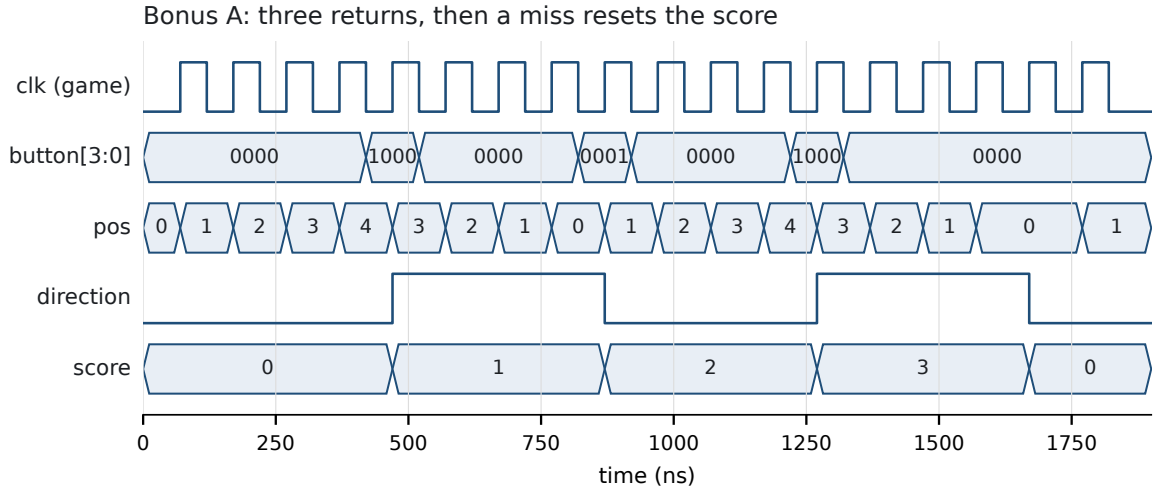


Figure 5. Bonus A: `pos` bounces 0→4→0, `direction` flips on each return.

5.2 B – Ping-pong, single clock domain

The second version keeps the whole design on the 50 MHz clock. A `pulse_generator` emits a one-cycle `tick` every max_count + 1 cycles (0.2 s at level 0, halving per level) and the game logic only moves the light when `tick` is high – a clock *enable* instead of a derived clock. Button edges are detected in the fast domain, so even a short press is registered. Reaching level 9 wins (all LEDs on). The testbench plays nine correct returns and reaches the win state, and measures the level-9 tick period: 20,001 cycles, as designed.

5.3 Observations

- Doubling the speed per level quickly exceeds human reaction time (≈0.2 s): in version A a step is shorter than 0.2 s from level 4, in version B from level 1. A gentler progression (e.g. ×1.25 per level) would make the later levels playable.
- In version B a miss is not penalised and repeated presses during one visit to an end each add a level. Counting at most one return per visit (and resetting on a miss, as in version A) would restore the intended rules.
- The push buttons are not debounced; a bouncing contact can register several edges. A small debounce counter or sampling on the slow tick would avoid this.

6 Implementation results

Table 1. Vivado post-implementation results (XC7Z010).

Design	LUTs	Flip-flops	WNS (ns)	Timing
Part 2 – LED flow	39	70	15.461	met
Part 3 – reaction game (block design)	28	32	16.215	met
Part 4 – LED flow clocked by the PS	40	70	15.246	met
Bonus A – ping-pong (block design)	49	44	15.189	met
Bonus B – ping-pong, single clock	78	48	13.266	met

7 Verification summary

Testbench	What is checked	Result
`tb_clk_led`	LED equals the clock	**PASS**
`tb_led_blink`	25 toggles, each after exactly one half-period	**PASS**
`tb_led_flow` (Parts 2, 4)	20 steps of the pattern; step time after the speed buttons	**PASS**
`tb_mini_game`	four hits → win; hit + miss → level 0	**PASS**
`tb_ping_pong`	three returns → score 3; miss → score 0	**PASS**
`tb_ping_pong_v2`	nine returns → win; tick period at level 9	**PASS**

The cleaned code contains no functional changes for this experiment; the block designs are provided as equivalent RTL top levels (`reaction_game_top`, `ping_pong_top`) together with the original `.bd` files.

8 Conclusion

A clock divider turns the 50 MHz oscillator into human-scale timing, and registers that change the divider at run time give speed control and level-dependent games. The block-design projects show how RTL modules are combined in IP Integrator, including feedback from the game logic to its own clock and, in Part 4, a PL clock supplied by the ARM processing system. The second ping-pong design illustrates the preferred style for FPGA logic: one clock domain with enable pulses.